《《第三章 SQL 注入漏洞与防护》》实验报告

任务 1：数据库与查询条件基础说明

1. 页面输入为什么与数据库查询有关

以 DVWA 的 SQL 注入模块为例，这个页面的功能是让用户输入一个用户 ID，然后查看该用户的详细信息。用户的所有信息都存储在后台的 MySQL 数据库中。所以这个功能本质上就是“接收用户输入→去数据库找对应数据→返回给用户”，页面输入自然就和数据库查询绑定在一起。

2. 最小业务链路

用户在前端输入 ID → 后台 PHP 代码接收输入 → 拼接成 SQL 查询语句 → MySQL 数据库执行查询 → 结果返回给 PHP → 页面渲染显示。

具体到 DVWA 里：我在输入框输入 1，后端就会生成类似 SELECT first_name, last_name FROM users WHERE user_id = '1' 的语句，然后数据库把 ID 为 1 的用户信息返回并显示在页面上。

3. 用户输入是如何进入查询条件的

用户输入是通过 PHP 的变量直接嵌入到 SQL 语句的字符串里的。比如后台代码可能是这样写的：

```
php
$id = $_GET['id'];
$query = "SELECT * FROM users WHERE user_id = '$id'";
```

这里我输入的 1 就直接被放到了 SQL 语句的引号中间，成为了查询条件的一部分。

4. 为什么这为 SQL 注入创造了可能性

因为代码没有把“用户输入的数据”和“SQL 命令本身”分开。用户输入的东西是当作“数据”还是当作“命令”，完全取决于输入内容里有没有能改变 SQL 结构的字符。一旦有人在输入里加入单引号、OR 等关键字，就能闭合前面的引号、追加新的查询逻辑，把原本的“查一条”变成“查全部”甚至“查别人的表”。这就是 SQL 注入产生的根本原因。

任务 2：DVWA SQL Injection Low 现象记录

实验页面：DVWA → SQL Injection → 难度等级选 Low

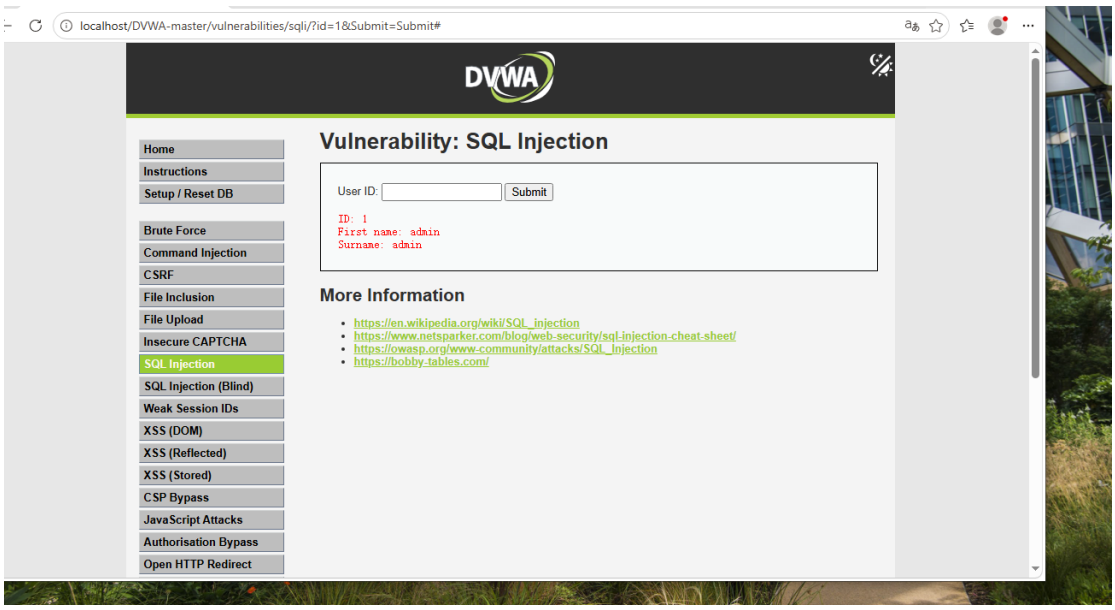
实验目标：观察普通输入和注入输入在页面返回结果上的差异，理解为什么这不像“正常查询变化”。

普通输入

输入：1

页面返回：ID: 1, First name: admin, Surname: admin

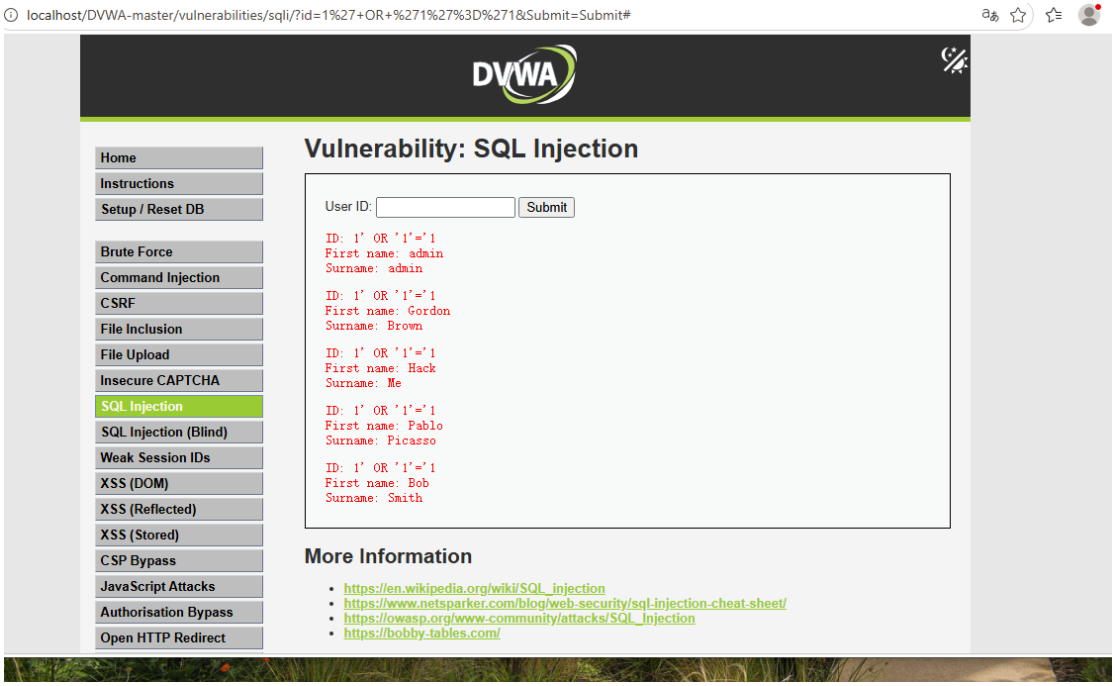
这是正常的查询结果，输入一个存在的用户 ID，系统返回对应的用户信息。



异常输入

输入：1' or '1'='1

页面返回：显示了数据库中所有用户的信息。



返回结果的差异

普通输入只返回了一条记录，而异常输入返回了多条记录，甚至把所有用户信息都列出来了。

为什么这种差异不像“正常查询变化”

如果我输入的是一个不存在的 ID，比如 999，页面应该返回“没有找到”或者空结果，这

才是正常查询变化。但现在我只加了一个单引号和一些 SQL 条件，结果却返回了所有用户数据。这说明输入内容不仅改变了“查询条件”，更是改变了“整个 SQL 语句的结构”。正常情况下输入框只是用来筛选具体的一条记录，但现在它让查询逻辑从“查一条”变成了“查全部”，这不是查询条件范围的变化，而是查询结构本身被篡改了。

判断支撑：这个现象说明输入的内容被直接拼接进了 SQL 语句，并且数据库执行了被篡改后的逻辑，证明了 SQL 注入漏洞确实存在。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户输入通过 URL 参数传递到服务器，服务器端 PHP 代码直接用字符串拼接的方式把输入变量嵌入到 SQL 语句模板中，形成完整的 SQL 语句后发送给数据库执行。这个过程就是：输入 → 拼接 → 解析 → 执行 → 回显。输入先变成 SQL 字符串的一部分，然后数据库解析器对整条语句进行语法分析，最后执行并返回结果。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

危险的不是单引号、or、union 这些字符本身，而是它们进入了 SQL 语句后改变了语句的语法结构。比如单引号正常情况下只是字符串的边界标记，但如果用户输入的单引号被拼进 SQL 语句后，它可以提前闭合前面的字符串，让后面的内容变成可执行的 SQL 代码。真正的问题是：输入内容本应只作为“数据”存在，却被当成了“代码”来解析执行。这就是“查询结构被污染”的含义——原本的查询逻辑被用户输入篡改了。

3. Low 与 Medium 的差异在哪里

Low 级别直接接收用户输入并拼接到 SQL 语句中，没有任何过滤。

Medium 级别在前端使用了 POST 方式提交，并且在 PHP 代码里对用户输入调用了 mysqli_real_escape_string()函数，这个函数会对单引号、双引号等特殊字符进行转义处理，让它们变成普通字符而不是 SQL 语法的一部分。

两者的差异在于：Low 完全没有控制，输入可以直接进入查询结构；Medium 试图在“输入”这一层做转义，让危险字符失效。但 Medium 的防护思路仍然是“把输入清理干净后再拼接”，并没有做到结构分离，所以如果遇到不依赖引号的注入方式（比如数字型注入），Medium 仍然可能被绕过。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

普通 SQL 注入：输入注入语句后，页面会直接返回数据库查询的结果，比如返回所有用户信息，攻击者可以直接从页面看到数据。

Blind SQL Injection：页面不会直接返回数据库内容，攻击者无法直接看到数据，但可以通过页面返回的“行为差异”来推断信息，比如页面是否正常显示、响应时间是否有变化、是否返回不同错误信息等。

两者的根因完全相同——都是输入进入了查询结构导致 SQL 逻辑被篡改。差异在于证

据形式不同：普通注入能看到返回数据的内容，盲注只能看到页面的状态或行为变化。
5. 为什么盲注和普通注入根因相同，但证据形式不同
因为漏洞产生的原因都是“查询结构被污染”，这是不变的。证据形式不同取决于网站对查询结果的“处理方式”：如果网站直接把数据库返回的内容显示出来，那就是普通注入；如果网站不显示数据内容，只返回一个“成功”或“失败”的页面，或者统一处理错误信息，那攻击者就只能通过页面行为差异来判断注入是否成功，这就是盲注的证据形式。
强制判断题
第一题：SQL 注入的根因更接近 B. 输入进入了不该进入的查询结构
理由：单引号、or、union 等字符本身并不危险，它们在正常的数据输入中是合法字符。真正导致注入的原因是这些输入内容被拼接进了 SQL 语句里，让数据库误把它们当成了 SQL 语法的一部分去解析执行。如果输入内容和数据查询结构是分离的，哪怕输入了单引号也不会产生注入。所以根因是“结构被污染”，不是“字符危险”。
第二题：Blind SQL Injection 和普通 SQL 注入的最大差异更接近 B. 证据形式不同
理由：两者的根因完全一样，都是输入进入了查询结构。区别只在于攻击者观察漏洞的方式不同——一个能直接看到数据内容，一个只能通过页面行为变化来推断。这不是漏洞本质上不同，而是证据表现的形式不同。

任务 4：SQL 注入防护清单与解释

1. 参数化查询

它解决什么问题：让用户输入只作为“数据”参数绑定到 SQL 语句中，而不是拼接到 SQL 字符串里，从根源上防止查询结构被污染。

它控制哪一层：根因控制（结构分离层）。参数化查询直接作用在 SQL 语句构造环节，把代码逻辑和用户数据彻底分开，数据库不会把输入内容当成 SQL 命令去解析。

它不能替代什么：不能替代输入校验和权限控制。即使 SQL 注入被防住了，用户输入仍可能包含恶意内容（比如 XSS 攻击脚本），数据库账号权限过大也可能造成数据泄露。参数化查询只解决注入问题，不解决其他安全问题。

2. 输入校验

它解决什么问题：在用户输入进入系统之前进行白名单验证，比如只允许数字的地方就只接受数字，只允许字母的地方拒绝特殊符号，减少攻击面。

它控制哪一层：暴露面控制（入口层）。输入校验在用户数据进入业务逻辑之前就做了过滤，让恶意输入尽量进不来。

它不能替代什么：不能替代参数化查询。因为校验规则可能被绕过或者有遗漏，只要

有一个输入点没校验到位，拼接的地方就可能出问题。它是第一道防线，但不能作为唯一防线。攻击手法不断变化，仅靠黑名单过滤特殊字符是不可靠的。

3. 错误处理

它解决什么问题：隐藏数据库返回的详细错误信息，避免攻击者通过报错内容推断数据库结构或判断注入是否成功。

它控制哪一层：后果范围控制（信息暴露层）。错误处理不阻止注入发生，但让攻击者拿不到有价值的反馈信息，增加攻击难度。

它不能替代什么：不能替代参数化查询和输入校验。因为即使错误信息被隐藏了，注入漏洞本身还存在，攻击者仍然可以通过盲注、时间延迟等其他方式利用漏洞。它只是增加了攻击成本，并没有修复漏洞本身。

4. 最小权限

它解决什么问题：限制数据库连接账号的权限，只赋予必要的操作权限，避免攻击者通过注入获取更高权限进行删改、拖库等破坏性操作。

它控制哪一层：后果范围控制（损失缩减层）。最小权限不阻止注入发生，但限制了注入成功后攻击者能做的事情。

它不能替代什么：不能替代参数化查询。因为即使权限很低，攻击者仍然可以通过注入窃取当前权限能访问的敏感数据。它只是降低了被攻击后的损失范围，并没有消除注入漏洞。

5. Web 应用防火墙（WAF）

它解决什么问题：在流量层检测和拦截常见的 SQL 注入攻击模式，通过规则匹配识别恶意请求并阻断，作为外围防护。

它控制哪一层：暴露面控制（网络层）。WAF 在网络流量进入应用服务器之前进行过滤，拦截已知的攻击特征。

它不能替代什么：不能替代代码层面的防护。因为 WAF 规则可以被绕过（比如编码绕过、分块传输等），攻击手法不断更新，WAF 只是辅助手段，不能作为唯一的防御措施。真正的安全要从代码层面解决。

总结：这五项措施分别作用在不同层面，谁也替代不了谁。参数化查询是唯一从根源解决问题的，控制的是“结构污染”；输入校验和 WAF 是减少攻击可能性的，控制的是暴露面；错误处理和最小权限是在漏洞存在的情况下降低损失、增加攻击难度的，控制的是后果范围。一个安全的系统应该多层防护配合使用，形成纵深防御。

收口项 1：SQL 注入因果链/防护关系图

环节	描述	对应防护/控制点	控制层级
1. 输入点	用户在 DVWA 输入框提交 ID	输入校验	暴露面控制
2. 查询构造	PHP 代码将输入拼接到 SQL 语句	参数化查询	根因控制
3. 数据库解析/执行	MySQL 执行被污染的查询语句	最小权限	后果控制
4. 结果回显	页面返回全部用户数据或报错	错误处理	后果控制
5. 根因判断	查询结构被用户输入改变	_____	_____

关系链说明

因果链从输入点开始，经过查询构造的拼接环节进入数据库执行，最终通过页面回显暴露出结果。防护措施要对应到不同环节：参数化查询直接切断“拼接”这个根因环节，输入校验减少攻击面，错误处理和最小权限在后果层兜底。四个措施缺一不可，但参数化查询是解决根因的关键。

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？

完全理解了为什么 SQL 注入的根因是“结构被污染”而不是某个字符危险。从我做的实验来看，1' OR '1'='1 能成功，是因为输入进入了 SQL 语句的结构层，改变了原本的查询逻辑，而不是因为单引号本身有魔法。

2. 我最容易混淆的一点是什么？

一开始没搞清楚 Low 和 Medium 到底本质区别在哪。后来想明白了：Low 是在拼接层没做任何事，Medium 是在输入层做过滤，但两者的根因都没解决，Medium 只是让攻击变难了。

3. 我本次作业中哪一段最需要教师复核？

任务 3 里关于盲注和普通注入“证据形式不同”的解释，我理解了概念，但不知道这句话在实际实验里是否表述得足够准确。

4. 如果重做一次，我会优先改哪一部分？

任务 2 的截图分析部分，我会把每个现象对应的判断写得更直接，比如“这个异常说明查询逻辑被改变了”，而不是只描述现象本身。

收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

本次作业使用了 AI 工具来：

· 理解作业要求和评分标准

· 梳理整体回答框架和结构

· 协助组织各任务答案的逻辑层次

2. 哪些判断是我自己复核后保留的

以下核心判断是我在实验和思考后确认保留的：

· SQL 注入的根因是“结构被污染”而非“字符危险”

· 普通注入与盲注的差异在证据形式而非根因

· 参数化查询对应根因控制，错误处理对应后果控制

· 所有强制判断题的答案及理由

3. 哪些内容我发现 AI 输出不准确并进行了修正

· AI 无法访问我的 DVWA 实验环境，所以任务 2 中的实验截图、输入记录、现象观察均为我亲自操作并记录

· AI 建议的具体输入 payload 我亲自测试验证后才采用

· 截图分析部分我根据自己的实验截图重新描述，确保与我的实际操作一致

4. 我最终如何确认结论和证据一一对应

提交前我逐项核对：

· 任务 2 的截图是否有对应的分析文字

· 每个截图分析是否明确说明了“这个现象支撑了什么判断”

· 任务 3 的机制分析是否能从任务 2 的实验中找到依据

· 防护措施的层级划分是否有逻辑支撑

所有结论都基于我自己在 DVWA 上亲手做的实验和截图，没有凭空得出的判断。

提交前自查清单（已全部核对）：

- 我在授权环境中完成实验
- 我的截图已脱敏
- 我说明了“正常查询变化”和“查询逻辑被改变”的区别
- 我把根因写成“结构被污染”，而不是只写“特殊字符”
- 我区分了普通 SQL 注入与盲注的证据差异
- 我把参数化查询、错误处理、最小权限放回不同控制层
- 我完成了关系图/关系表
- 我填写了 AI 输出审核与人工复核记录《《